

CM02 - Bash Scripting

Builtins, processes, variables, scripts, conditions, loops, functions

Louis Ledoux

2026-07-05

Table of contents

1 Command Implementation and Processes	2
1.1 Main Path	2
1.2 Worked Example	3
1.3 Anecdote Or Motivation	3
1.4 Check Question	4
1.5 Backup Index	4
1.6 Backup: CM 2 Bash scripting	7
1.7 Backup: Command implementation	7
1.8 Backup: Builtin Commands	8
1.9 Backup: Builtin Commands	8
1.10 Backup: Process management: Execution	9
1.11 Backup: Process management: Changing grounds	10
1.12 Backup: Process management	11
1.13 Backup: Process management: Active processes	12
1.14 Backup: Process management: Control execution	13
2 Shell Variables and Scripts	13
2.1 Main Path	13
2.2 Worked Example	14
2.3 Anecdote Or Motivation	14
2.4 Check Question	15
2.5 Backup Index	15
2.6 Backup: Shell Variables	18
2.7 Backup: Shell Variables: Scope	19
2.8 Backup: Shell Variables: Scope example	19
2.9 Backup: Predefined shell variables	20
2.10 Backup: Scripts: Structure	20
2.11 Backup: Scripts: Execution	21
2.12 Backup: Command-line arguments	22
2.13 Backup: Variables	23
2.14 Backup: Read input	24
2.15 Backup: Print to output	25
2.16 Backup: Arithmetic operations	25
2.17 Backup: Arithmetic operations	26
3 Conditions	26
3.1 Main Path	27
3.2 Worked Example	27
3.3 Anecdote Or Motivation	28
3.4 Check Question	28
3.5 Backup Index	29

3.6 Backup: Condition expressions: test	31
3.7 Backup: Condition expressions: test operators	32
3.8 Backup: Exit status	33
3.9 Backup: Arithmetic operations	34
3.10 Backup: Boolean control operators	34
3.11 Backup: Conditions	35
3.12 Backup: AND and OR control operators: Example	36
3.13 Backup: Conditions: if, elif, else	37
3.14 Backup: Conditions: if : Examples	37
3.15 Backup: Conditions: if : Examples	38
3.16 Backup: Conditions: case	39
3.17 Backup: Conditions: case: Example	40
4 Iterations	41
4.1 Main Path	41
4.2 Worked Example	42
4.3 Anecdote Or Motivation	42
4.4 Check Question	43
4.5 Backup Index	43
4.6 Backup: Iterations: for	46
4.7 Backup: Iterations: for: Examples	47
4.8 Backup: Iterations: while	48
4.9 Backup: Iterations: while: Examples	49
4.10 Backup: Iterations: until	50
4.11 Backup: Iterations: until: Examples	51
4.12 Backup: Controlling loop commands	52
4.13 Backup: Controlling loop commands: Example	52
4.14 Backup: Controlling loop commands: Example	53
5 Functions	54
5.1 Main Path	54
5.2 Worked Example	55
5.3 Anecdote Or Motivation	55
5.4 Check Question	56
5.5 Backup Index	56
5.6 Backup: Functions	57
5.7 Backup: Functions: Example	58

1 Command Implementation and Processes

Live pacing: about 8 min

Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

1.1 Main Path

- Start from the question: When does the shell execute commands, and when does it just transform text?
- Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- Keep only this slide on the horizontal path during the live lecture.

- CM 2 Bash scripting

#50

1.2 Worked Example

- Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- Representative source slide: 50
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

- CM 2 Bash scripting

#50

1.3 Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Process management Control execution

- End a running process
- Global method:
- kill PID
- Foreground execution in current shell (through SIGNALS)
- CTRL + C (Interrupt)
- CTRL + D (EOF)
- CTRL + \ (Quit)
- Pause a process
- CTRL + Z
- Resume
- Foreground (fg)
- Background (bg)

#58

1.5 Backup Index

Original slides 50-58

- CM 2 Bash scripting

#50

slide 50

Command implementation

- Builtin
- Implemented in the code of the shell
- They affect the internal state of the shell (ex. cd)
- External
- Implemented as a separate code
- Shell interprets the command as a request to load and run the program that implements the command
- Stored in /bin

#51

slide 51

Builtin Commands

#52

slide 52

Builtin Commands

#53

slide 53

Process management: Execution

- Process: The running instance of a program, e.g.:
 - Execution of a command
 - Execution of your program
- Foreground execution: Interactive processes
 - Connected to a shell terminal
 - User can send input
- Background execution: Non-Interactive or automated processes
 - Not connected to a terminal
 - User cannot send input until the process is moved to the foreground

#54

slide 54

Process management Changing grounds

- Run in Background: &
- Ex: gedit &
- From FG to BG:
- Ex: gedit
- Press CTRL + Z (puts process in idle state)
- bg
- From BG to FG:
- Ex: fg %jobid
- (jobid is not PID. It is a unique number given by the shell, e.g. when process goes to BG)

#55

slide 55

Anchors: CM 2 Bash scripting; Command implementation; Builtin Commands; Process management: Execution; Process management: Changing grounds; Process management

1.6 Backup: CM 2 Bash scripting

Original slide 50

- CM 2 Bash scripting

#50

1.7 Backup: Command implementation

Original slide 51

- Builtin
- Implemented in the code of the shell

- They affect the internal state of the shell (ex. cd)
- External
- Implemented as a separate code
- Shell interprets the command as a request to load and run the program that implements the command
- Stored in /bin

Command implementation

- Builtin
- Implemented in the code of the shell
- They affect the internal state of the shell (ex. cd)
- External
- Implemented as a separate code
- Shell interprets the command as a request to load and run the program that implements the command
- Stored in /bin

#51

1.8 Backup: Builtin Commands

Original slide 52

Builtin Commands

#52

1.9 Backup: Builtin Commands

Original slide 53

1.10 Backup: Process management: Execution

Original slide 54

- Process: The running instance of a program, e.g.:
- Execution of a command
- Execution of your program
- Foreground execution: Interactive processes
- Connected to a shell terminal
- User can send input
- Background execution: Non-Interactive or automated processes
- Not connected to a terminal
- User cannot send input until the process is moved to the foreground

Process management: Execution

- Process: The running instance of a program, e.g.:
- Execution of a command
- Execution of your program
- Foreground execution: Interactive processes
- Connected to a shell terminal
- User can send input
- Background execution: Non-Interactive or automated processes
- Not connected to a terminal
- User cannot send input until the process is moved to the foreground

#54

1.11 Backup: Process management: Changing grounds

Original slide 55

- Run in Background: `&`:
- Ex: `gedit &`
- From FG to BG:
- Ex: `gedit`
- Press `CTRL + Z` (puts process in idle state)
- `bg`
- From BG to FG:
- Ex: `fg %jobid`
- (jobid is not PID. It is a unique number given by the shell, e.g. when process goes to BG)

Process management Changing grounds

- Run in Background: &
- Ex: gedit &
- From FG to BG:
- Ex: gedit
- Press CTRL + Z (puts process in idle state)
- bg
- From BG to FG:
- Ex: fg %jobid
- (jobid is not PID. It is a unique number given by the shell, e.g. when process goes to BG)

#55

1.12 Backup: Process management

Original slide 56

- Process: The running instance of a program, e.g.:
- Execution of a command
- Execution of your program
- Serial execution: ;
- All commands are executed
- New prompt when last command finishes
- Ex: who | sort ; date
- Parallel execution: &
- All commands are executed
- A new prompt is directly available (execution at the background)
- Ex: pr junk | lpr &

Process management

- Process: The running instance of a program, e.g.:
- Execution of a command
- Execution of your program
- Serial execution: ;
- All commands are executed
- New prompt when last command finishes
- Ex: who | sort; date
- Parallel execution: &
- All commands are executed
- A new prompt is directly available (execution at the background)

#56

1.13 Backup: Process management: Active processes

Original slide 57

- Print the catalogue of the running processes: ps
- -a: Select all processes associated with a terminal.
- PID: Unique identity of the process
- TTY: Console created the process
- TIME: Total execution time (sec)
- CMD: Command executed

Process management: Active processes

- Print the catalogue of the running processes: ps
- -a: Select all processes associated with a terminal.
- PID: Unique identity of the process
- TTY: Console created the process
- TIME: Total execution time (sec)
- CMD: Command executed

```
angie@achilleas:~/Downloads$ ps
  PID TTY          TIME CMD
 12190 pts/1        00:00:00 bash
 14414 pts/1        00:00:00 vi
 14447 pts/1        00:00:00 evince
 14454 pts/1        00:00:00 ps
```

#57

1.14 Backup: Process management: Control execution

Original slide 58

- End a running process
- Global method:
- kill PID
- Foreground execution in current shell (through SIGNALS)
- CTRL + C (Interrupt)
- CTRL + D (EOF)
- CTRL + \ (Quit)
- Pause a process
- CTRL + Z
- Resume
- Foreground (fg)
- Background (bg)

Process management Control execution

- End a running process
- Global method:
- kill PID
- Foreground execution in current shell (through SIGNALS)
- CTRL + C (Interrupt)
- CTRL + D (EOF)
- CTRL + \ (Quit)
- Pause a process
- CTRL + Z
- Resume
- Foreground (fg)
- Background (bg)

#58

2 Shell Variables and Scripts

Live pacing: about 8 min

Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

2.1 Main Path

- Start from the question: When does the shell execute commands, and when does it just transform text?
- Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- Keep only this slide on the horizontal path during the live lecture.

Shell Variables

- Initial values: defined in system and user configuration file
- Use uppercase by convention
- Definition of variables based on the type (bash shell)
- String variables: MYVAR='value' (NO SPACES around =)
- Integer variables: declare -i MYVAR
- Constant variables: readonly DATA=value
- Array variables: declare -a MYARRAY MYARRAY[0]='one'; MYARRAY[1]=5
- Use of variables: \$
- E.g. \$MYVAR
- Lifetime of variables

#59

2.2 Worked Example

- Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- Representative source slide: 64
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Scripts Execution

```
No need for permissions:  
bash path_to_script/myscriptsh  
bash command is executed  
path_to_script/myscriptsh is the command-line argument  
Provide eXecute permissions and execute:  
chmod +x myscriptsh  
./myscriptsh  
Shell forks itself and uses a system call (e.g. execve) to make the OS execute the file in the  
forked process.  
OS checks the file's permissions
```

#64

2.3 Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Arithmetic operations

- Arithmetic operations with integers
- let (without spaces as it is an initialisation)
- expr (with spaces in arguments)

```

alain@alain:~$ let a=1; echo $a
1
alain@alain:~$ let a++; echo $a
2
alain@alain:~$ expr $a + 1 ; echo $a
3
alain@alain:~$ expr $a + 1 ; echo $a
4
alain@alain:~$
  
```

#70

2.5 Backup Index

Original slides 59-70

Shell Variables

- Initial values: defined in system and user configuration file
- Use uppercase by convention
- Definition of variables based on the type (bash shell)
- String variables: MYVAR="value" (NO SPACES around =)
- Integer variables: declare -i MYVAR
- Constant variables: readonly DATA=value
- Array variables: declare -a MYARRAY MYARRAY[0]='one'; MYARRAY[1]=5
- Use of variables: \$
- E.g. \$MYVAR
- Lifetime of variables

#59

slide 59

Shell Variables: Scope

- Global: Public variable seen by all subshells
- Set export attitude for the variable: export
- Print global variables: env, printenv
- Local variables: Seen by current shell
- Print list of local and global variables: set

#60

slide 60

Shell Variables: Scope example

```
angie@achilleas:~$ myvar="hello"
angie@achilleas:~$ set | grep myvar
myvar=hello
angie@achilleas:~$ echo "$$"
12190
angie@achilleas:~$ bash
angie@achilleas:~$ echo "$$"
15752
angie@achilleas:~$ set | grep myvar
angie@achilleas:~$ env | grep myvar
angie@achilleas:~$ exit
exit
angie@achilleas:~$ export myvar="hello"
angie@achilleas:~$ bash
angie@achilleas:~$ echo "$$"
15825
angie@achilleas:~$ set | grep myvar
myvar=hello
angie@achilleas:~$ █
```

#61

slide 61

Predefined shell variables

#62

slide 62

Scripts: Structure

Program consisting of commands
Use an editor:
vi (vim), emacs, sublime, gedit, ...
1st line defines the shell to be used:
#!/bin/bash (for bash shell)
Write program commands
Commands can be separated by:
a new line
Semicolon;
Split a command to 2 or more lines: \ (backslash)

#63

slide 63

```
No need for permissions:  
bash path_to_script/myscriptsh  
bash command is executed  
path_to_script/myscriptsh is the command-line argument  
Provide eXecute permissions and execute:  
chmod +x myscriptsh  
./myscriptsh  
Shell forks itself and uses a system call (e.g. execve) to make the OS execute the file in the  
forked process  
OS checks the file's permissions
```

slide 64

anchors: Shell Variables; Shell Variables: Scope; Shell Variables: Scope example; Predefined shell variables; Scripts: Structure; Scripts: Execution

2.6 Backup: Shell Variables

Original slide 59

- Initial values: defined in system and user configuration file
- Use uppercase by convention
- Definition of variables based on the type (bash shell)
- String variables: MYVAR="value" (NO SPACES around =)
- Integer variables: declare -i MYVAR
- Constant variables: readonly DATA=value
- Array variables: declare -a MYARRAY MYARRAY[0]="one"; MYARRAY[1]=5
- Use of variables: \$
- E.g. \$MYVAR
- Lifetime of variables:
- End of session - termination of shell
- Deleted by user: unset VARNAME

Shell Variables

- Initial values: defined in system and user configuration file
- Use uppercase by convention
- Definition of variables based on the type (bash shell)
- String variables: MYVAR="value" (NO SPACES around =)
- Integer variables: declare -i MYVAR
- Constant variables: readonly DATA=value
- Array variables: declare -a MYARRAY MYARRAY[0]='one'; MYARRAY[1]=5
- Use of variables: \$
- E.g. \$MYVAR
- Lifetime of variables

#59

2.7 Backup: Shell Variables: Scope

Original slide 60

- Global: Public variable seen by all subshells
- Set export attitude for the variable: export
- Print global variables: env, printenv
- Local variables: Seen by current shell
- Print list of local and global variables: set

Shell Variables: Scope

- Global: Public variable seen by all subshells
- Set export attitude for the variable: export
- Print global variables: env, printenv
- Local variables: Seen by current shell
- Print list of local and global variables: set

#60

2.8 Backup: Shell Variables: Scope example

Original slide 61

```
angie@achilleas:~$ myvar="hello"
angie@achilleas:~$ set | grep myvar
myvar=hello
angie@achilleas:~$ echo "$$"
12190
angie@achilleas:~$ bash
angie@achilleas:~$ echo "$$"
15752
angie@achilleas:~$ set | grep myvar
angie@achilleas:~$ env | grep myvar
angie@achilleas:~$ exit
exit
angie@achilleas:~$ export myvar="hello"
angie@achilleas:~$ bash
angie@achilleas:~$ echo "$$"
15825
angie@achilleas:~$ set | grep myvar
myvar=hello
angie@achilleas:~$ █
```

#61

2.9 Backup: Predefined shell variables

Original slide 62

#62

2.10 Backup: Scripts: Structure

Original slide 63

```
Program consisting of commands:
Use an editor:
vi (vim), emacs, sublime, gedit, ...
1st line defines the shell to be used:
```

```
#!/bin/bash (for bash shell)
Write program commands
Commands can be separated by:
a new line
Semicolon: ;
Split a command to 2 or more lines: \ (backslash)
For comments: #
Save file with extension .sh
The script lines (commands) are interpreted and NOT compiled
```

Scripts Structure

```
Program consisting of commands:
Use an editor:
vi (vim), emacs, sublime, gedit ...
1st line defines the shell to be used:
#!/bin/bash (for bash shell)
Write program commands
Commands can be separated by:
a new line
Semicolon: ;
Split a command to 2 or more lines: \ (backslash)
```

#63

2.11 Backup: Scripts: Execution

Original slide 64

```
No need for permissions:
bash path_to_script/myscript.sh
bash command is executed
path_to_script/myscript.sh is the command-line argument.
Provide eXecute permissions and execute:
chmod +x myscript.sh
./myscript.sh
Shell forks itself and uses a system call (e.g. execve) to make the OS execute the file in
the forked process.
OS checks the file's permissions
Program loader looks at the file to find how to execute it
(#! for scripts and then which interpreter to use: /bin/bash).
New process: attention to the variables and functions
```

Scripts Execution

```
No need for permissions:  
bash path_to_script/myscriptsh  
bash command is executed  
path_to_script/myscriptsh is the command-line argument.  
Provide eXecute permissions and execute:  
chmod +x myscriptsh  
./myscriptsh  
Shell forks itself and uses a system call (e.g. execve) to make the OS execute the file in the  
forked process.  
OS checks the file's permissions
```

#64

2.12 Backup: Command-line arguments

Original slide 65

```
Arguments can be passed along with the script execution:  
./myscript.sh arg1 ... argN  
The arguments are seen as specific variables in the script code:  
$0: Name of the script  
$1: 1st argument  
$N: Nth argument  
$#: Number of arguments  
$*: All the arguments passed to the script, as a string  
$@: Behaves like $* except that when the arguments are quoted, they are broken up properly if  
there are spaces in them  
Maximum length of arguments is defined by operating system and measured in Kilobytes.  
getconf ARG_MAX
```

Command-line arguments

Arguments can be passed along with the script execution:

```
./myscript.sh arg1 ... argN
```

The arguments are seen as specific variables in the script code:

\$0: Name of the script

\$1: 1st argument

\$N: Nth argument

\$#: Number of arguments

\$*: All the arguments passed to the script as a string

@@: Behaves like \$* except that when the arguments are quoted, they are broken up properly if there are spaces in them

#65

2.13 Backup: Variables

Original slide 66

Initialization: `nomvar=expression`

ATTENTION! No space around =

expression:

Value

Reference to a variable

``command`` (replaced with the result to the stdout)

Use: `$nomvar`

Print current directory

```
y=`pwd`
```

```
echo $y
```

Print the first two arguments of a script

```
y="$1 and $2"
```

```
echo $y
```

Variables

Initialization: `nomvar=expresion`
ATTENTION! No space around =
expression:

#66

2.14 Backup: Read input

Original slide 67

- We can ask the user to provide some value(s) to the variables
- `read var-name1 var-name2 ... var-nameN`
- Reads a line from the stdin and affects the first value to `var-name1`, the second value to `var-name2`, etc.
- Returns 0, if the read was successful
- We can print a message to the user:
- `read -p "Enter value:" var`
- If variable is not specified, then the default is used:
- `$REPLY`

Read input

- We can ask the user to provide some value(s) to the variables
- `read var-name1 var-name2 ... var-nameN`
- Reads a line from the stdin and affects the first value to `var-name1`, the second value to `var-name2`, etc.
- Returns 0, if the read was successful
- We can print a message to the user:
- `read -p "Enter value:" var`
- If variable is not specified, then the default is used:
- `$REPLY`

#67

2.15 Backup: Print to output

Original slide 68

- Basic print to stdout: echo
- Strings
- Variables
- Wildcards characters
- Can have several lines

Print to output

- Basic print to stdout: echo
- Strings
- Variables
- Wildcards characters
- Can have several lines

```
angie@achilleas:~/Entertainment$ echo hello
hello
angie@achilleas:~/Entertainment$ echo *
movies Music photos
angie@achilleas:~/Entertainment$ echo print '@' "user"
print @ user
angie@achilleas:~/Entertainment$ echo 'hell
> o there'
hell
o there
```

#68

2.16 Backup: Arithmetic operations

Original slide 69

- No need to define the variables as integers
- Arithmetic operations with integers
- arithmetic expansion: expression evaluation and substitution with the result
- `$((...))`: Portability among shells
- `((...))`

Arithmetic operations

- No need to define the variables as integers
- Arithmetic operations with integers
- arithmetic expansion: expression evaluation and substitution with the result
- `$(...)`: Portability among shells
- `{(...) }`

[illegible]

#69

2.17 Backup: Arithmetic operations

Original slide 70

- Arithmetic operations with integers
- let (without spaces as it is an initialisation)
- expr (with spaces in arguments)

Arithmetic operations

- Arithmetic operations with integers
- let (without spaces as it is an initialisation)
- expr (with spaces in arguments)

```

1 3 4
miniloo~ akritika~ a~
miniloo~ akritika~ let a=a~1; echo $a
4
miniloo~ akritika~ let a+= echo $a
5
miniloo~ akritika~ a+= expr $a + 1; echo $a
6
miniloo~ akritika~ a+= expr $a~1; echo $a
6~
miniloo~ akritika~ █

```

#70

3 Conditions

Live pacing: about 8 min

Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

3.1 Main Path

- Start from the question: When does the shell execute commands, and when does it just transform text?
- Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- Keep only this slide on the horizontal path during the live lecture.

Condition expressions: test

- A condition expression is:
 - a set of commands that are executed
 - The test command exits with the exit status determined by the condition_expression
- test condition_expression
- [condition_expression]
- [[condition_expression]]
- Similar to single brackets, but more powerful
- ((arithmetic condition_expression))
- (command)

- SPACES ARE REQUIRED !!

#71

3.2 Worked Example

- Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- Representative source slide: 73
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Exitstatus

Syntax

\$?

Function

If the command ends correctly (TRUE): exits with value 0 (exit 0)

- ATTENTION !!

If there was an error (FALSE): exits with value 1 (exit 1)

Commands that do nothing except return an exit status of:

true: Return an exit status of zero

false: Return an exit status of one

```
$ true; echo $?  
$ 0  
$ false; echo $?  
$ 1
```

#73

3.3 Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

3.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Conditions: case: Example

```
#!/bin/bash  
read -p "Enter a command: " command  
case $command in  
all | ALL )  
echo "Display all files.."  
ls -la ;;  
list | LIST )  
echo "Display all non-hidden files.."  
ls -ll ;;  
*)  
echo "Invalid choice"  
esac
```

#82

3.5 Backup Index

Original slides 71-82

Condition expressions: test

- A condition expression is:
 - a set of commands that are executed
 - The test command exits with the exit status determined by the condition_expression
- test condition_expression
- [condition_expression]
- [[condition_expression]]
- Similar to single brackets, but more powerful
- ((arithmetic condition_expression))
- (command)

- SPACES ARE REQUIRED !!

#71

slide 71

Condition expressions: test operators

#72

slide 72

Exit status

Syntax

\$?

Function

If the command ends correctly (TRUE): exits with value 0 (exit 0)

- ATTENTION !!

If there was an error (FALSE): exits with value 1 (exit 1)

Commands that do nothing except return an exit status of:

true: Return an exit status of zero

false: Return an exit status of one

```
$ true; echo $?  
$ 0  
$ false; echo $?  
$ 1
```

#73

slide 73

Arithmetic operations

- Kahoot time !
- (4 Q)

#74

slide 74

Boolean control operators

- Syntax (AND): `-a or &&`
- `command1 && command2`
- Function:
 - `command2` is executed if, and only if, `command1` returns an exit status of zero (TRUE).
- Syntax (OR): `-o or ||`
- `command1 || command2`
- Function:
 - `command2` is executed if, and only if, `command1` returns a non-zero exit status (FALSE).
 - The exit status of AND and OR lists is the exit status of the last command executed in the list
- Syntax (NOT): `!`

#75

slide 75

Conditions

- Kahoot time !
- (4 Q)

#76

slide 76

Anchors: Condition expressions: test; Condition expressions: test operators; Exit status; Arithmetic operations; Boolean control operators; Conditions

3.6 Backup: Condition expressions: test

Original slide 71

- A condition expression is:
- a set of commands that are executed
- The test command exits with the exit status determined by the `condition_expression`

- test condition_expression
- [condition_expression]
- [[condition_expression]]
- Similar to single brackets, but more powerful
- ((arithmetic condition_expression))
- (command)
- SPACES ARE REQUIRED !!

Condition expressions: test

- A condition expression is:
- a set of commands that are executed
- The test command exits with the exit status determined by the condition_expression
- test condition_expression
- [condition_expression]
- [[condition_expression]]
- Similar to single brackets, but more powerful
- ((arithmetic condition_expression))
- (command)

- SPACES ARE REQUIRED !!

#71

3.7 Backup: Condition expressions: test operators

Original slide 72

3.8 Backup: Exit status

Original slide 73

Syntax

\$?

Function

If the command ends correctly (TRUE): exits with value **0** (exit **0**)

If there was an error (FALSE): exits with value **1** (exit **1**)

Commands that **do** nothing except **return** an exit status of:

true: Return an exit status of zero

false: Return an exit status of one

- ATTENTION !!

```
$ true ; echo $?  
$ 0  
$ false ; echo $?  
$ 1
```

Exitstatus

Syntax

\$?

Function

If the command ends correctly (TRUE): exits with value 0 (exit 0)

- ATTENTION !!

If there was an error (FALSE): exits with value 1 (exit 1)

Commands that do nothing except return an exit status of:

true: Return an exit status of zero

false: Return an exit status of one

```
$ true; echo $?  
$ 0  
$ false; echo $?  
$ 1
```

#73

3.9 Backup: Arithmetic operations

Original slide 74

- Kahoot time !
- (4 Q)

Arithmetic operations

- Kahoot time !
- (4 Q)

#74

3.10 Backup: Boolean control operators

Original slide 75

- Syntax (AND): -a or &&
- command1 && command2
- Function:

- command2 is executed if, and only if, command1 returns an exit status of zero (TRUE).
- Syntax (OR): -o or ||
- command1 || command2
- Function:
- command2 is executed if, and only if, command1 returns a non-zero exit status (FALSE).
- The exit status of AND and OR lists is the exit status of the last command executed in the list
- Syntax (NOT): !

Boolean control operators

- Syntax (AND): -a or &&
- command1 && command2
- Function:
- command2 is executed if, and only if, command1 returns an exit status of zero (TRUE).
- Syntax (OR): -o or ||
- command1 || command2
- Function:
- command2 is executed if, and only if, command1 returns a non-zero exit status (FALSE).
- The exit status of AND and OR lists is the exit status of the last command executed in the list
- Syntax (NOT): !

#75

3.11 Backup: Conditions

Original slide 76

- Kahoot time !
- (4 Q)

Conditions

- Kahoot time !
- (4 Q)

#76

3.12 Backup: AND and OR control operators: Example

Original slide 77

```
$ true || echo "echo executed"
$ false || echo "echo executed"
echo executed
$ true && echo "echo executed"
echo executed
$ false && echo "echo executed"
$
```

AND and OR control operators: Example

```
$ true || echo 'echoexecuted'
$ false || echo 'echoexecuted'
echoexecuted
$ true && echo 'echoexecuted'
echoexecuted
$ false && echo 'echoexecuted'
$
```

#77

3.13 Backup: Conditions: if, elif, else

Original slide 78

- Syntax:
- if [condition_expression_1] ; then
- 1st set of commands
- elif [condition_expression_2]
- then
- 2nd set of commands
- else
- 3rd set of commands
- fi
- Function
- The if checks if the condition_expression_1 returns TRUE
- 1st set of commands is executed
- Else, if the condition_expression_2 returns TRUE
- 2nd set of commands is executed
- Otherwise, 3rd set of commands is executed
- SPACES for [] ARE REQUIRED !!

Conditions: if, elif, else

```
- Syntax
- if [condition_expression_1]; then
- 1st set of commands
- elif [condition_expression_2]
- then
- 2nd set of commands
- else
- 3rd set of commands
- fi
- Function
```

- SPACES for [] ARE REQUIRED !!

#78

3.14 Backup: Conditions: if : Examples

Original slide 79

```
#!/bin/bash
read -p "Enter a filename: " filename
```

```

if [ ! -w "$filename" ]; then
echo "File $filename is not writable"
exit 1
elif [ ! -r "$filename" ]; then
echo "File $filename is not readable"
exit 1
else
echo "File $filename is writable and readable"
fi

```

Conditions if: Examples

```

#!/bin/bash
read -p "Enter a filename: " filename
if [ ! -w "$filename" ]; then
echo "File $filename is not writable"
exit 1
elif [ ! -r "$filename" ]; then
echo "File $filename is not readable"
exit 1
else
echo "File $filename is writable and readable"
fi

```

#79

3.15 Backup: Conditions: if : Examples

Original slide 80

```

#!/bin/bash
TMPFILE="diff.out"
diff $1 $2 > $TMPFILE
if [ ! -s "$TMPFILE" ]; then
echo "Files are the same"
else
more $TMPFILE
fi
if [ -f "$TMPFILE" ]; then
rm $TMPFILE
fi

```

Conditions if: Examples

```
#!/bin/bash
TMPFILE='diff.out'
diff $1 $2 > $TMPFILE
if [ !-s "$TMPFILE" ]; then
echo "Files are the same"
else
more $TMPFILE
fi
if [ -f "$TMPFILE" ]; then
rm $TMPFILE
fi
```

#80

3.16 Backup: Conditions: case

Original slide 81

- Syntax:
- case \$variable in
- pattern 1)
- 1st set of commands ;;
- pattern 2 | pattern 3)
- 2nd set of commands ;;
- *)
- Nth set of commands ;;
- esac
- Function
- If the value of variable is pattern 1, the 1st set of commands is executed, if it is pattern 2, the 2nd set of commands is executed etc.
- Otherwise (*) -> Nth set of commands

Conditions: case

- Syntax
- case \$variable in
- pattern 1)
- 1st set of commands;;
- pattern 2 | pattern 3)
- 2nd set of commands;;
- *)
- Nth set of commands;;
- esac
- Function

#81

3.17 Backup: Conditions: case: Example

Original slide 82

```
#!/bin/bash
read -p "Enter a command: " command
case $command in
all | ALL )
echo "Display all files..."
ls -la ;;
list | LIST)
echo "Display all non-hidden files..."
ls -ll ;;
*)
echo "Invalid choice"
esac
```

```
#!/bin/bash
read -p "Enter a command: " command
case $command in
all | ALL)
echo "Display all files.."
ls -la ;;
list | LIST)
echo "Display all non-hidden files.."
ls -ll ;;
*)
echo "Invalid choice"
esac
```

4 Iterations

Live pacing: about 8 min

Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

4.1 Main Path

- Start from the question: When does the shell execute commands, and when does it just transform text?
- Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- Keep only this slide on the horizontal path during the live lecture.

Iterations: for

Syntax:
for variable in list_of_values
do
set of commands done
Function
In each iteration variable takes a value from the list_of_values
The set of commands is execute for this value
list_of_values can be:
Strings: L3info L3miage M1info
Numbers: counting: {1..10..2}

#83

4.2 Worked Example

- Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- Representative source slide: 83
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Iterations: for

Syntax:
for variable in list_of_values
do
set of commands done
Function
In each iteration variable takes a value from the list_of_values
The set of commands is execute for this value
list_of_values can be:
Strings: L3info L3miage M1info
Numbers: counting: {1..10..2}

#83

4.3 Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

4.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Controlling loop commands: Example

```
- x is 1. Most numbers get here, except 4.
- x is 2. Most numbers get here, except 4.
- x is 3. Most numbers get here, except 4.
- x is 4.
- x is 5. Most numbers get here, except 4.
- x is 6. Most numbers get here, except 4.
- x is 7. Most numbers get here, except 4.
- We have finished the loop now.
- At the end, x is now 7.
```

#91

4.5 Backup Index

Original slides 83-91

Iterations: for

```
Syntax:
for variable in list_of_values
do
set of commands done
Function
In each iteration variable takes a value from the list_of_values
The set of commands is execute for this value
list_of_values can be:
Strings: L3info L3miage M1info
Numbers counting: {1..10..2}
```

#83

slide 83

Iterations: for: Examples

```
#!/bin/bash
for file in f1 f2 f3
do
cp $file $1/$file
done
```

```
#!/bin/bash
for i in 6 3 1 2
do
echo $i
done | sort -n
```

#84

slide 84

Iterations: while

- Syntax
- while
- [condition_expression]
- do
- set of commands
- done
- Function
- As long as the condition_expression returns TRUE
- The set_of_commands is executed
- Otherwise, the iteration ends

#85

slide 85

Iterations: while: Examples

```
#!/bin/bash
i=1
while [ $i -lt 10 ]
do
echo $i
((i++))
done
```

```
#!/bin/bash
while true
do
echo 'alive...'
sleep 3s
done
```

#86

slide 86

Iterations: until

- Syntax
- until
- [condition_expression]
- do
- set of commands
- done
- Function
- Execute 1st set of commands
- As long as the condition_expression is NOT true (while condition_expression is FALSE)
- The set_of_commands is executed

#87

slide 87

```
#!/bin/bash
i=1
until [ $i -ge 10 ]
do
echo $i
((i++))
done
```

slide 88

Anchors: Iterations: for; Iterations: for: Examples; Iterations: while; Iterations: while: Examples; Iterations: until; Iterations: until: Examples

4.6 Backup: Iterations: for

Original slide 83

Syntax:

```
for variable in list_of_values
do
```

set of commands done

Function

In each iteration variable takes a value from the list_of_values

The set of commands is execute for this value

list_of_values can be:

Strings: L3info L3miage M1info

Numbers: counting: {1..10..2}

enumerating: 1 3 5

Command line arguments: \$*

File names: filename1 filename2

command output: \$(ls /tmp/*)

Iterations: for

Syntax:

for variable in list_of_values

do

set of commands done

Function

In each iteration variable takes a value from the list_of_values

The set of commands is execute for this value

list_of_values can be:

Strings: L3info L3miage M1info

Numbers: counting: {1..10..2}

#83

4.7 Backup: Iterations: for: Examples

Original slide 84

```
#!/bin/bash
for file in f1 f2 f3
do
cp $file $1/$file
done
```

```
#!/bin/bash
for i in 6 3 1 2
do
echo $i
done | sort -n
```

Iterations: for: Examples

```
#!/bin/bash
for file in f1 f2 f3
do
cp $file $1/$file
done
```

```
#!/bin/bash
for i in 6 3 1 2
do
echo $i
done | sort -n
```

#84

4.8 Backup: Iterations: while

Original slide 85

- Syntax:
- while
- [condition_expression]
- do
- set of commands
- done
- Function
- As long as the condition_expression returns TRUE
- The set_of_commands is executed
- Otherwise, the iteration ends

Iterations: while

- Syntax
- while
- [condition_expression]
- do
- set of commands
- done
- Function
- As long as the condition_expression returns TRUE
- The set_of_commands is executed
- Otherwise, the iteration ends

#85

4.9 Backup: Iterations: while: Examples

Original slide 86

```
#!/bin/bash
i=1
while [ $i -lt 10 ]
do
echo $i
((i++))
done
```

```
#!/bin/bash
while true
do
echo "alive..."
sleep 3s
done
```

```
#!/bin/bash
i=1
while [ $i -lt 10 ]
do
echo $i
((i++))
done
```

```
#!/bin/bash
while true
do
echo 'alive...'
sleep 3s
done
```

4.10 Backup: Iterations: until

Original slide 87

- Syntax:
- until
- [condition_expression]
- do
- set of commands
- done
- Function
- Execute 1st set of commands
- As long as the condition_expression commands is NOT true (while condition_expression is FALSE)
- The set_of_commands is executed
- Otherwise the iteration ends

Iterations: until

- Syntax
- until
- [condition_expression]
- do
- set of commands
- done
- Function
- Execute 1st set of commands
- As long as the condition_expression commands is NOT true (while condition_expression is FALSE)
- The set_of_commands is executed

#87

4.11 Backup: Iterations: until: Examples

Original slide 88

```
#!/bin/bash
i=1
until [ $i -ge 10 ]
do
echo $i
((i++))
done
```

Iterations: until: Examples

```
#!/bin/bash
i=1
until [ $i -ge 10 ]
do
echo $i
((i++))
done
```

#88

4.12 Backup: Controlling loop commands

Original slide 89

- Stop with current loop iteration: break
- Continue with the next iteration: continue
- Terminate script: exit

Controlling loop commands

- Stop with current loop iteration: break
- Continue with the next iteration: continue
- Terminate script: exit

#89

4.13 Backup: Controlling loop commands: Example

Original slide 90

```
#!/bin/bash
for x in 1 2 3 4 5 6 7 8 9
do
echo -n "x is $x. "
if [ "$x" -eq "4" ]; then
continue
fi
echo "Most numbers get here, except 4."
if [ "$x" -eq "7" ]; then
break
fi
done
echo "We have finished the loop now."
echo "At the end, x is now $x."
```

Controlling loop commands: Example

```
#!/bin/bash
for x in 1 2 3 4 5 6 7 8 9
do
echo -n 'x is $x. '
if [ "$x" -eq 4 ]; then
continue
fi
echo "Most numbers get here, except 4."
if [ "$x" -eq 7 ]; then
break
fi
done
echo "We have finished the loop now."
echo "At the end, x is now $x."
```

#90

4.14 Backup: Controlling loop commands: Example

Original slide 91

- x is 1. Most numbers get here, except 4.
- x is 2. Most numbers get here, except 4.
- x is 3. Most numbers get here, except 4.
- x is 4.
- x is 5. Most numbers get here, except 4.
- x is 6. Most numbers get here, except 4.
- x is 7. Most numbers get here, except 4.
- We have finished the loop now.
- At the end, x is now 7.

Controlling loop commands: Example

- x is 1. Most numbers get here, except 4.
- x is 2. Most numbers get here, except 4.
- x is 3. Most numbers get here, except 4.
- x is 4.
- x is 5. Most numbers get here, except 4.
- x is 6. Most numbers get here, except 4.
- x is 7. Most numbers get here, except 4.
- We have finished the loop now.
- At the end, x is now 7.

#91

5 Functions

Live pacing: about 8 min

Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

5.1 Main Path

- Start from the question: When does the shell execute commands, and when does it just transform text?
- Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- Keep only this slide on the horizontal path during the live lecture.

Functions

```
Syntax
function name[()]
{
  list of commands
  [return]}
Function
Functions have to be defined in the beginning of the script
They may have or not arguments
Arguments and return values can be of any type
The variables defined inside the function are GLOBAL. They have to be defined LOCAL, if we want
```

#92

5.2 Worked Example

- Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- Representative source slide: 92
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Functions

```
Syntax
function name[()]
{
  list of commands
  [return]}
Function
Functions have to be defined in the beginning of the script
They may have or not arguments
Arguments and return values can be of any type
The variables defined inside the function are GLOBAL. They have to be defined LOCAL, if we want
```

#92

5.3 Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

5.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Functions: Example

```
#!/bin/bash
outside="a global variable"
function mine() {
  local inside="this is local"
  echo $outside
  echo $inside
  outside="a global with new value"
}
echo $outside
mine
echo $outside
echo $inside
```

- a global variable
- a global variable
- this is local
- a global with new value

#93

5.5 Backup Index

Original slides 92-93

Functions

Syntax

```
function name{[]}
```

{

list of commands

[return]}

Function

Functions have to be defined in the beginning of the script

They may have or not arguments

Arguments and return values can be of any type

The variables defined inside the function are GLOBAL. They have to be defined LOCAL, if we want

#92

slide 92

Functions: Example

```
#!/bin/bash
outside='a global variable'
function mine() {
  local inside='this is local'
  echo $outside
  echo $inside
  outside='a global with new value'
}
echo $outside
mine
echo $outside
echo $inside
```

- a global variable
- a global variable
- this is local
- a global with new value

#93

slide 93

Anchor: Functions; Functions: Example

5.6 Backup: Functions

Original slide 92

Syntax

```
function name[()]
{
  list of commands;
  [return]}
```

Function

Functions have to be defined in the beginning of the script

They may have or not arguments

Arguments and **return** values can be of any type

The variables defined inside the function are GLOBAL. They have to be defined LOCAL, **if** we want to reduce their scope

Functions

```
Syntax
function name{[]
{
list of commands
[return]}
Function
Functions have to be defined in the beginning of the script
They may have or not arguments
Arguments and return values can be of any type
The variables defined inside the function are GLOBAL. They have to be defined LOCAL, if we want
```

#92

5.7 Backup: Functions: Example

Original slide 93

```
#!/bin/bash
outside="a global variable"
function mine() {
local inside="this is local"
echo $outside
echo $inside
outside="a global with new value"
}
echo $outside
mine
echo $outside
echo $inside
```

- a global variable
- a global variable
- this is local
- a global with new value

Functions: Example

```
#!/bin/bash
outside="a global variable"
function mine() {
  local inside="this is local"
  echo $outside
  echo $inside
  outside="a global with new value"
}
echo $outside
mine
echo $outside
echo $inside
```

- a global variable
- a global variable
- this is local
- a global with new value

#93